



Chapter 2

Foundations of

Quantitative Trading

Quantitative trading is a mix of art and science. In this chapter, we seek to introduce some of the core concepts to comfortably understand the examples in later chapters. These are in no way comprehensive, and due to quantitative trading's artistic nature, many parts are reasonably opinionated, but we'll endeavor to paint a balanced picture. If you're a professional quant or familiar with QuantConnect, feel free to skip this chapter.

Research Process

The classic quant research process flows from research, backtesting, parameter optimization, paper trading (out of sample), and live trading. Researchers following this path tend to reduce their time spent on ideas that may not result in profitable signals, eliminating ideas quickly in research before testing them in a highly accurate backtesting environment for merit once the realities of trading costs are accounted.

Research

A Jupyter notebook (qnt.co/book-cloud-research) provides a space for rapid exploration of ideas in an iterative way. Large datasets can be loaded into memory and plotted easily to visualize signals and test their effectiveness. This allows researchers to quickly explore ideas with a few lines of pandas code. Generally, this research applies vector operations on in-memory datasets, so care should be taken to avoid look-ahead bias.

Backtesting

Backtesting (qnt.co/book-cloud-backtesting) provides you with an accurate representation of your strategy performance, with many details of the market modeled, including trade fees, spreads, slippage, and interest penalties. Like live trading, data from a moment of time is injected into your strategy code bar by bar, allowing you to define how you will set up algorithm state and handle corporate events like splits and dividends. By carefully abstracting code, your research and backtesting code can translate directly to live trading with almost no changes.

Parameter Optimization

A parameter (qnt.co/book-cloud-optimization) is a variable in your strategy that controls the outcome of your backtesting. Try to minimize the number of parameters whenever possible to reduce the likelihood of overfitting. With 5–10 parameters, it is easy to make any strategy perform well in-sample. All strategies have parameters, from implicit ones in your date range and starting capital, to explicit ones in your indicators or signal definition. Your goal at this phase is to test a strategy's sensitivity to its parameters. This robustness testing can give you confidence that the strategy will perform in many environments and that you have not cherry-picked a good set of variables.

Paper and Live Trading

Deploying a strategy to live trading (qnt.co/book-cloud-live-trading) lets you ensure the algorithm signal works out of sample and can highlight potential issues in mechanical aspects of the algorithm, such as processing times, order fill delays, or data delays. Many of these issues don't come to light until after the first trade has been placed, so some practitioners believe it is best to jump straight from idea to live trading with a small amount of capital.

Testing and Debugging Tools

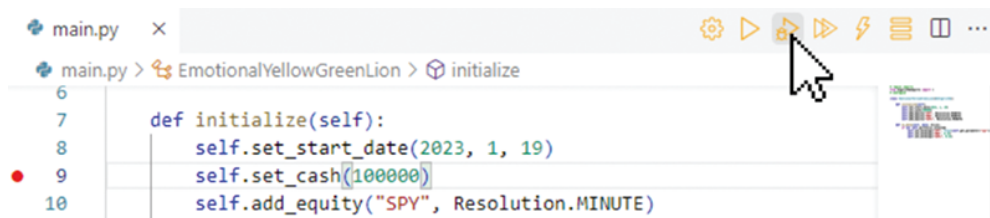
Building a robust algorithmic trading strategy is a challenging endeavor, and even seasoned practitioners rarely get it right on the first try. Using the right tools, you can quickly debug issues and deploy your strategy. In

addition, by adopting an iterative build-test, you can dramatically reduce the complexity of the errors encountered. In this section, we'll review the tools and coding processes to build strategies quickly.

There are four key tools for designing and debugging.

Debuggers

Debugging (qnt.co/book-cloud-debugging) should be the first step to understanding errors in backtesting code execution. Debuggers pause program execution to inspect the state of variables or execute code snippets to test executions. To trigger debugging in QuantConnect, click the border next to the line of code where you'd like the program to pause, then press the backtest button with the small bug icon (see [Figure 2.1](#)).



[Figure 2.1](#) Running backtests in QuantConnect Cloud with debugging mode.

Separate from the IDE debugger tool, QuantConnect can stream messages to the cloud console environment during a backtest or live trading. This is a crude way to confirm that signals are being triggered and get a sense of your strategy execution.

```
self.debug(f"Signal triggered, placing limit order: {limit}")
```

Logging

Log (qnt.com/book-logging) statements are strings recorded during backtests. It is best to record key decision moments because backtests can replay millions of data events. A log statement placed in the wrong place can generate millions of lines of text that are difficult to analyze. Take care when placing the log to keep them readable. Unlike streamed debug messages, log statements are permanent and can be referred to later if needed.

```
self.log(f"Signal triggered, placing limit order: {limit}")
```

Charting

Plots help to visualize models and help you understand edge cases for debugging. They can quickly reveal outliers in data and the signal's distribution of values.

When generating custom plots, you should first develop an intuitive guess about what you should see before you draw it. This can help you reject the results and debug the underlying issues. For example, a plot of an asset's price standard deviations should be between -4 and +4, with any outliers occurring during extreme market conditions, such as during the COVID-19 pandemic in March 2020.

In QuantConnect, there is a powerful custom charting application program interface (API, [qnt.com/book-charting](https://www.quantconnect.com/docs/v2/quant-connect-api/charting)). Default plots can be created with a single line of code or customized ones with two or three lines if the chart is defined first.

```
# Create a line plot with default options
self.plot("deviations", "AAPL", asset_deviation)

# Instantiate with custom colors and series type
chart = Chart("deviations")
chart.add_series( Series("AAPL", SeriesType.SCATTER, "$",
                        Color.GREEN, ScatterMarkerSymbol.TRIANGLE)
)
self.plot("deviations", "AAPL", asset_deviation)
```

Object Store

The QuantConnect Object Store ([qnt.co/book-object-store](https://www.quantconnect.com/docs/v2/quant-connect-api/object-store)) is a key-value data store for low-latency storage and retrieval of information. Objects can be stored and retrieved there programmatically across all the layers of QuantConnect technology (research, backtest, optimization, and live trading).

During a backtest, you can build large objects you'd like to analyze and store them for later analysis. This can be helpful when the objects are too

large to plot, or when you'd like to perform analysis across many backtests.

```
self.object_store.save("key", "value")      # Save a string to object store
string_data = self.object_store.read("key")  # Retrieve the object

self.object_store.save_bytes("key", bytes)   # Store a serialized object
byte_data = self.object_store.read_bytes("key") # Retrieve byte data array
```

It can be useful to pair this with the end of the algorithm event handler to write the object for analysis at the end of your strategy's backtest.

```
# At the end of algo, write data to object store under project id folder
def on_end_of_algorithm(self):
    self.object_store.save(f"{self.project_id}/backtest", self._data)
```

Coding Process

The most common mistake beginners make is writing hundreds of lines of code without running a single backtest. As long as you're using backtests to verify the *mechanics* of your implementation plan and not tune the value of a parameter, they are an excellent tool for building and debugging your strategy. Using backtesting to tune the strategy parameters risks overfitting, which we'll cover later in this chapter.

Using an iterative build process of a “build-test,” you can add one layer at a time to your strategy and then backtest it over a short window of time, verifying it behaves as you expect with debug statements, or verifying the order times and quantities match your expectations. At each layer, take the time to write clean, commented code with any necessary abstractions. This can feel clunky and slow initially, but the time saved in debugging a complex strategy can be enormous.

Time and Look-ahead Bias

Financial data comes in two different “shapes” according to the period it covers: point values or period values. As a backtest progresses, data is emitted in a precise sequence to prevent seeing future data points before they should be available to strategy code. This is done to prevent look-

ahead bias—a form of quant research error caused by having knowledge of the future in an algorithm.

Point data only has a single timestamp. Examples include a trade tick, a measurement of weather, or a customer transaction. Period values have two timestamps: a start time and an end time of the period they cover. Examples include a daily bar for stocks, opening at 9:30 am and closing at 4 pm ([Figure 2.2](#)).

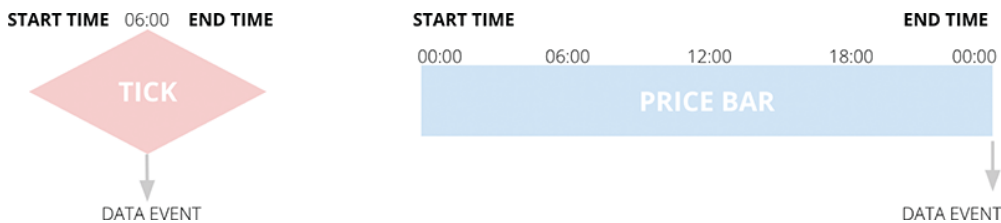


Figure 2.2 A data event occurs at the end of the data consolidation period. Tick data does not have a consolidation period, so data events occur when a new tick arrives.

In QuantConnect, period values are represented by the time and `end_time` data properties.

```
tick.time           # Instant of time trade or quote occurred
tradebar.time       # Start of the price bar
tradebar.end_time   # End of the price bar
```

Look-ahead Bias

Look-ahead bias occurs when knowledge of future events is used in an algorithm, causing the performance to look better in backtesting than it would in out-of-sample performance. It is very easy to fall into this bias. Some examples include the following:

- Designing strategies based on knowledge of recent major market movements.
- Timestamping data at the start time of a period rather than when it was available, which may be several days later. For example, government data representing “May-2024” likely won’t be available until June 10, the following month.
- Filling trades based on the close price of daily bars, which occurs when the market is closed.
- Using a large language model (LLM) trained and released in 2024 on a backtest that starts in 2018.

Market Hours and Scheduling

Every asset class has a set of market hours (qnt.co/book-market-hours) to describe when they are open for trading. This can be as simple as crypto, which is 24 hours a day, or as complex as futures markets that open and shut multiple times within a day. Most markets include market holidays when the market is closed entirely or shortened market hours around Christmas or other holidays. The collection of days and hours when a market is open for transactions is called a market calendar.

With robust time handling, you can model a market calendar in your algorithm. QuantConnect accurately accounts for these market hours for all supported asset classes. You can schedule code (qnt.co/book-scheduled-events) to run based on the market status using the scheduling API in QuantConnect:

```
# At the end of trading week, trigger rebalance function 30 minutes after
# market open - accounting for market holidays and trading hours.
symbol = Symbol.create('SPY', SecurityType.EQUITY, Market.USA)
self.schedule.on(
    self.date_rules.week_end(symbol),
    self.time_rules.after_market_open(symbol, 30),
    self._rebalance
)
```

For AI models, it's imperative to retrain them on recent data. This training is ideally done frequently to adjust to the latest market conditions. Using the built-in train method, you can schedule a function callback premarket and launch a long-running training method to refresh your model.

```
self.train(self.date_rules.every(DayOfWeek.SUNDAY),
           self.time_rules.at(8,0), self._training_method)
```

Strategy Styles

Most strategies are variants of four core ideas: momentum, reversion, scalping, or arbitrage. Momentum strategies use signals, such as technical indicators, sentiment, or news, to invest with the larger up-and-down swings of the market—attempting to exit before the trends lose momentum. Reversion strategies seek turning points in an asset's rapid rise and

bet it will return to historical mean prices or industry benchmarks.

Scalping strategies look for tiny fluctuations in asset prices and enter and exit quickly for small gains. Arbitrage strategies typically buy and sell the same or similar assets in different locations to benefit from small variations in the asset price.

Trading Signals

Strategies typically fall into two types of signal categories: a binary flag or a continuous signal. These signals are part of a strategy's alpha or factor creation.

Continuous signals are numeric and indicate the strength or rank of the idea being tested. Examples of this could include calculating a factor score based on the price-earnings (PE) ratio of a universe of assets. Assets could be assigned a score, and then allocated capital based on the score's strength.

Binary signals are discrete, generally signaling long, flat, or short investment directions. They could use the same data but only generate a trade signal when reaching some target criteria—for example, only emitting a signal when the PE ratio of an asset is less than five. More natural applications of discrete signals arise from event trading, such as the approval of a new drug release for a pharmaceutical company.

There are no fixed rules around these categories, and you can build hybrids together. For example, you could convert a binary news event announcing the FDA approval of a drug into a continuous signal, by calculating the probability that the drug has been approved based on the pre-market trials and other emerging public information. Once the final announcement is made, the probability would go to 100%. This allows your strategy to take positions before the final announcement.

Allocating Capital

Using these signals, traders then decide how to allocate their buying power for the best performance. There are three basic styles of capital allocation: discrete trade/bet placements, continuous portfolio allocations, or tactical allocations (a hybrid of these approaches). The allocation is typically the responsibility of a portfolio construction system.

Classic quantitative finance focuses on portfolio allocators' desire to keep capital invested at all times, following the old adage "time in market is more important than timing the market." They focus on continuous signals that generate a ranking of assets and create a diversified portfolio with the best assets possible. A whole field of portfolio allocation science, and most notably, Modern Portfolio Theory, emerged seeking to carefully weight portfolio assets to improve portfolio performance. An example of this could be ranking companies in the S&P500 by fundamental performance and allocating to the top 20 companies, with the allocation sizes inversely proportional to their trailing volatility. These strategies are best measured by alpha, beta, and Sharpe ratio, and benchmarked to the major indices.

```
# Rebalance portfolio to allocation by weights
targets = [PortfolioTarget("AAPL", 0.5), PortfolioTarget("MSFT", 0.5)]
self.set_holdings(targets)
```

Discrete trading strategies scan markets for specific criteria, and then enter trades with sizing and risk allocations independent of other holdings in the portfolio. The entry and exit signals are typically discrete (short -1, flat, or long 1). Because this is closer to how humans trade, it is a common bridge for professional traders automating their trading strategies. The performance of such strategies is often uncorrelated with wider markets and benchmarked by absolute returns—a target percent return per year. For risk control, more advanced versions of discrete trading use a modified Kelly criterion, treating asset trades as bets with individual payoffs and risk criteria. These strategies are commonly measured by win-rate, expectancy, and Sharpe ratio. Quantitatively minded discrete traders tend to put on many bets across a wide universe of assets to smooth their overall performance.

```
# Manually create and route orders for individual stocks or strategies
long_straddle = OptionStrategies.straddle(symbol, strike, expiry)
self.buy(long_straddle, 50)

# Volatility adjusted order position size with stops
trading_range = self.atr("SPY", 30)
quantity = risk_capital / trading_range.current.value
self.market_order("SPY", quantity)
self.stop_market_order("SPY", -quantity, data["SPY"].close - trading_range)
```

Tactical asset allocation is like classic portfolio allocation, in that it aims to be invested at all times but adjusts its allocation according to strategy signals. This could be small reweights of existing portfolio items or a complete replacement of the portfolio contents. An example of this is a strategy allocating to primary indices during bullish markets but reducing position sizes in equities and allocating to bonds during market volatility. These strategies are typically benchmarked to major indices and measured by alpha, beta to indices, and Sharpe ratio.

Regimes and Portfolios of Strategies

Striving for perfection across all market regimes is often a risky path that leads to overfitting. Unless your strategy is market agnostic—like a high-frequency trading strategy profiting from exchange rebates and routing trades, or a market-neutral strategy profiting from arbitrage opportunities—you must embrace that there will likely be drawdowns in your strategy. Many of the world’s top-performing funds, such as Renaissance Technologies or World Quant, use a “many alpha” approach with complex portfolio management systems that decide how to weigh the signals of their millions of alphas.

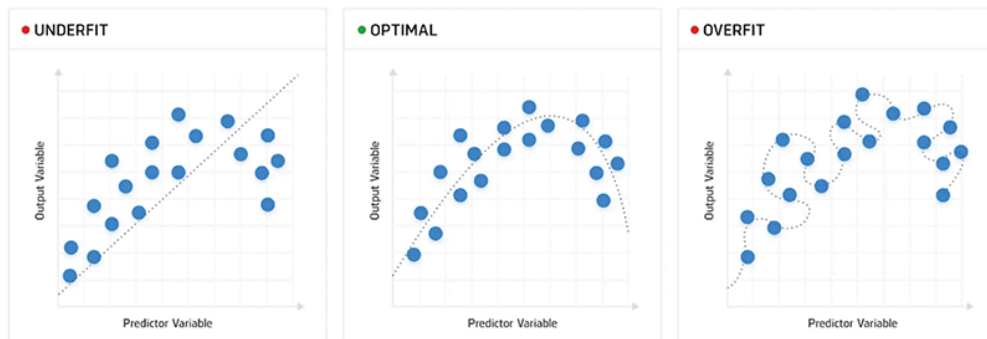
Bullish, strongly trending markets might best fit long-only, levered, trend-following strategies seeking to capitalize on the strong markets. During market turbulence, such as uncertainty about elections or interest rate hikes, the markets may tend to “range”—trading up and down within a narrow band of values until more clarity arrives. In these periods, mean-reversion strategies may perform better, buying the local price minima and selling the local price maxima. Finally, during bear markets, a portfolio manager may select defensive strategies like bonds or short-biased equities, and algorithms that react faster to market news due to the higher volatility in bear markets. Planning and building a suite of these strategies helps ensure you have the right portfolio bias during market regime changes.

Although price is quite challenging to predict, volatility tends to have momentum. Investors often panic together and cause rapid price movements. Some quant’s design strategies focused on high- or low-volatility environments using options or shorting volatility.

Parameter Sensitivity Testing and Optimization

Parameters (gnt.co/book-cloud-optimization) are variables in every algorithm that impact its performance. These variables can be very subtle, and you often need to carefully review your strategy code to identify them all—even very subtle properties like starting date and initial capital can have substantial impacts. One of the most common errors of quantitative research is overfitting a strategy to past data.

Overfitting occurs when a function is too closely fit to a limited training data set. Overfitting can occur in your trading algorithms if you have many parameters or select parameter values that worked very well in the past but are sensitive to small changes in their values. In these cases, your algorithm will likely be fine-tuned to fit the detail and noise of the historical data to the extent that it negatively impacts the *live* performance of your algorithm. [Figure 2.3](#) shows examples of underfit, optimally fit, and overfit functions.



[Figure 2.3](#) Overfitting and underfitting a model leads to poor predictions on samples outside of the training dataset.

A strategy that may be optimally fitted would not necessarily have the smoothest equity curve. It would have parameter choices that showed the strongest performance over a wide range of potential values. To lower the chances of overfitting, we recommend following the following steps:

1. Remove

Remove as many parameters as possible from your strategy. Ideally, the parameter can simply be removed entirely, such as eliminating a signal that isn't contributing significantly to the strategy returns. For machine learning, this can be done with PCA (discussed in later chapters) to transform the original dataset into its key components.

2. Replace

When it's not possible to remove a parameter, seek to replace it with a probability, natural (real-world) reason, or a data source.

Imagine your strategy was looking for news releases every hour at minute zero (10:00 am), seeking to aggregate the last hour of sentiment and rebalancing the portfolio every hour. Rather than optimizing to see which minute *off* the hour (e.g., 10:03, 10:06, 10:07 am) gives the best performance, it would be more natural to **remove** this parameter and build a rolling window of sentiment and perform the analysis continuously. Alternatively, you could **replace** this parameter by finding a data source for the news feeds and perform the sentiment analysis when there is a news release for your assets—a real-world replacement for a parameter.

Once you've built the new sentiment analysis technique, it would be tempting to introduce another parameter, such as rebalancing when you see a sentiment jump of $\pm 10\%$. Instead of introducing this arbitrary parameter, you could replace it by calculating a rolling sentiment standard deviation score. This could emit a constant sentiment signal strength 'stdev/4' that a portfolio construction system uses in its allocations decisions—dynamically scaling the allocation according to sentiment. Although this unfortunately introduces another subtle parameter—the lookback window period for the standard deviation—it is at least a dynamically changing probability, rather than an arbitrary fixed target score.

3. Reduce

Finally, and potentially the hardest step to take, is to avoid the lure of backtesting while adjusting parameters to improve performance. Whenever possible, build your strategy on a small window of time, testing its mechanics and preserving as much recent data as possible for the out-of-sample test. Before you put the algorithm into forward trading, you can test it on out-of-sample data, but aim to only use this data two or three times before discarding an idea.

Parameter Sensitivity Testing

QuantConnect offers a powerful grid search parameter testing tool to quickly explore all possible combinations of a backtest's parameters. This is somewhat dangerous in the wrong hands, and with great power comes great responsibility. In QuantConnect, parameters for this tool are set in the side panel of each project and loaded into your code as shown here:

```
# retrieve stored parameter
parameter_value = self.get_parameter("parameterName")
```

We recommend that our tool be used for *parameter sensitivity testing*—exploring the decay in performance across a range of parameter values. Your algorithm is likely robust if total performance is similar across a wide range of parameter values. If you see a concentration of returns on a narrow range of parameter values, it is likely an artifact in the data, or a few isolated trades are dramatically improving your performance. In recent history, the biggest cause of these outsized returns is a perfectly timed short of the March 2020 COVID crash, a 6-sigma global event unlikely to happen again soon. If your combination of parameters is shorting COVID-2020, you're likely overfitting.

The Sharpe ratio does not detect these hidden outliers underlying the equity curve, as it only considers the total mean and variance of the returns and relies on a normal distribution of returns in the data. Given enormous empirical evidence showing that returns *do not* follow a normal distribution, Bailey and De Prado (2012) proposed the Probabilistic Sharpe Ratio (PSR) as a probability measure calculated from a sample of the returns. Using the discrete daily or trade returns, the PSR informs us of the probability that the estimated Sharpe ratio is greater than a chosen benchmark (i.e., whether the estimate is statistically significant).

In QuantConnect, this is calculated and included in the backtest results using a benchmark Sharpe ratio of 1.0. You can interpret the PSR scores generated as a percent probability that the true Sharpe ratio exceeds 1.0—i.e., a PSR score of 77% indicates a 77% chance your strategy has a Sharpe ratio above 1.0.

Margin Modeling

Every strategy starts with a capital allocation and seeks to deploy it as efficiently as possible. To do this, algorithms must manage their buying power well to open their desired positions (qnt.com/book-buying-power).

Margin is a calculation of a trading account's maximum allowed purchasing power. It is based on its assets on hand (including cash), the account type, and your brokerage. There are a few different ways to model margin defined by the SEC and FINRA, but ultimately, it's a line of credit. This makes it your broker's decision on how much leverage it will provide based on the market risk and collateral in account.

The margin available and the total margin used at any time are available as properties of the portfolio object in your QuantConnect strategy. These properties calculate the portfolio-wide margin across all asset types.

```
self.portfolio.margin_remaining    # Buying power available for allocation
self.portfolio.total_margin_used   # Buying power allocated on holdings
```

Equities

Equities have three main forms of margin available for investors:

Cash Accounts—Buying power is restricted to cash on hand. After a trade, it can take up to three days to settle the cash and transfer ownership of the shares. In QuantConnect, this can be specified with the brokerage model.

```
# Set cash account and all broker models
self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE,
                        AccountType.CASH)
```

Margin Accounts—Brokers can offer up to 2x overnight or 4x intraday of additional buying power with margin accounts. Order settlement is also instant, allowing fast redeployment of capital. Most of the examples in this book rely on leverage margin accounts.

```
# Set cash account and all broker models
self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE,
                        AccountType.MARGIN)
```

Portfolio Margin—A dynamic risk-based estimate of the margin available based on holding volatility and liquidity. If you have safe, cash-like assets, they will be useful collateral for the broker and your available margin will remain high. Portfolio margin accounts settle instantly and offer up to 6.7x your initial cash as buying power. QuantConnect does not have portfolio margin modeling yet, but you can achieve a crude approximation of it by setting leverage to 7.

```
self.add_equity("SPY", leverage=7)
```

Equity Options

Equity option contracts provide leveraged exposure to the equity assets. They can be used in combinations as betting instruments for many investment theses, including insurance, high volatility, low volatility, ranging, or trending strategies. The notional value of the contract is 100x the list price of the option contract as it represents 100 underlying equity shares.

Option strategies are combinations of option contracts that reduce the total exposure of an investment. Dozens of investment strategies are available, and QuantConnect models 16 of them. They can be accessed with the shortcut helpers, as shown in the following example. This example adjusts its asset trading strategy according to the forecast volatility, selecting the closest expiry date when it's a low volatility short strategy:

```
if low_volatility:
    contracts = OptionStrategies.short_straddle(option_symbol, strike, min(expiri
else:
    contracts = OptionStrategies.straddle(option_symbol, strike, max(expiries))

# Using the OptionStrategies helper, get the contracts to buy
order_tickets = self.buy(contracts, 1)
```

Generally, when buying an option, risk is constrained to the purchase of the option contracts. Margin requirements increase when selling options, which require potentially settling the agreement with 100 shares of the underlying per contract in the event of a loss. This is where hedged strategies shine, by providing a reduction in the total risk exposure of the strategy.

In QuantConnect, the hedging margin requirements are calculated automatically for you, allowing you to focus on the alpha of the strategy and make better use of available capital.

Futures

Future contracts charge a fixed amount per contract, called the initial margin, and then require a minimum amount to be kept in the brokerage account to keep the contract open, called the maintenance margin. As with equities, if the account falls below the minimum requirements, the exchange or broker can liquidate the position.

Each contract has a varying initial and maintenance margin requirement that the exchange periodically updates according to contract price and volatility movements. The notional value for a future contract (total value controlled) is the unit asset price multiplied by the contract multiplier. The contract multiplier was originally the typical unit of sale for the commodity (e.g., 100 barrels of oil), but now with the rise of financial futures, it is typically a scaling factor to provide contracts with similar exposure to other listed futures.

In QuantConnect, all the prices, contract multipliers, and initial and maintenance margin requirements are modeled to provide accurate point-in-time buying power. This happens automatically, but if you'd like to explicitly get the number of contracts or shares you can buy or sell with a fixed capital allocation, you can use the following method:

```
# Fee-adjusted quantity of contracts for 20% buying power
contract = canonical_future.mapped
quantity = self.calculate_order_quantity(contract, 0.2)
```

Diversification and Asset Selection

Diversification is when you invest in a variety of assets to earn better overall returns. If you combine assets correctly, you can reduce your overall portfolio risk, as the drawdowns of one investment will be offset by gains in another.

“Diversification is the only free lunch in investing.”

— Harry Markowitz

With QuantConnect, you can add multiple stocks and asset types to your portfolio and explore the benefits of harnessing diversification. The most straightforward form of diversification is selecting assets with different industries, customer bases, and risk-return profiles. By choosing fundamentally diverse investments, you can improve your portfolio's chances of surviving market crashes. On QuantConnect, you can select stocks based on their industry to diversify within US equities or even invest across one of the 11 asset classes we support. Additionally, through our exchange traded fund (ETF) support, you can trade bonds and international market ETFs.

```
# Purchase a portfolio of targets, processing orders intelligently
self.set_holdings([
    PortfolioTarget("SPY", 0.6), PortfolioTarget("BND", 0.4)
])
```

To improve your portfolio diversification, you can measure asset correlations and add them to your portfolio when they are relatively uncorrelated with your existing holdings. Correlation is a measure of how similar a new asset is to your existing portfolio returns. A value of 1.0 indicates the new asset moves identically to the current portfolio, and -1.0 shows it moves precisely opposite the current portfolio. A value of 0 means the asset moves independently.

```
def beta(self, asset_returns, benchmark_returns):
    covariance = np.cov(asset_returns, benchmark_returns)[0, 1]
```

```
variance = np.var(benchmark_returns)
return covariance / variance
```

A common mistake is adding stocks to a portfolio that seem independent but move together when the market experiences volatility. Seemingly uncorrelated assets often become correlated in market corrections, so a critical review of historical correlations is essential to surviving future downturns.

Fundamental Asset Selection

In QuantConnect, the selection of assets is called universe selection (qnt.co/book-universe-selection). Universe selection is the picking of the assets for your strategy based on codified criteria. Codifying the selection removes human bias from the decision of assets. Precisely, what filter you apply depends on your strategy investment goals, but the common root of universe selection is avoidance of *selection* or *survivorship* bias through the use of a system or algorithm to include the assets in your algorithm.

Selection bias is an incomplete or unscientific selection of constituents for use in your strategy. The most common form of this is selecting an asset simply due to personal attachment to the brand or company.

Survivorship bias is a type of selection bias where the universe of assets for an investment is limited to those that are still available today. The most common form of this bias would be using today's top performing assets as your universe for a multi-year backtest. Over the last 10–20 years, many companies have failed and been delisted. By selecting assets listed today, you are implicitly applying knowledge of the future.

For example, in hindsight, it is easy to tell Apple or NVIDIA were breakout successes for investors, but at the time, picking them out from thousands of other technology stocks was a much harder task.

Even in the definition of the universe selection function, it is easy to overfit your strategy. If you've hardcoded revenue or profitability filters for your asset universe, keep in mind they could be favorably screening assets. Once the core selection criteria have been defined, you should return to test the sensitivity of the new parameters.

In the following QuantConnect example, we perform selection on corporate fundamental data. In the fundamental object, approximately 900 properties of each company are available for selection.

```
def select(self, fundamental):
    filtered = [f for f in fundamental if not np.isnan(f.valuation_ratios.pe_ratio)]
    sorted_by_pe_ratio = sorted(filtered, key=lambda f: f.valuation_ratios.pe_ratio)
    return [f.symbol for f in sorted_by_pe_ratio[:10]]
```

Scheduled universe selection reapplies the universe filters at a fixed cadence based on the market calendar. This can increase the algorithm's backtesting speed and reduce portfolio churn by selecting assets for longer. By default, universe selection runs daily, which is too frequent for many strategies.

```
# Re-run selection at the start of every trading calendar month of SPY.
self.universe_settings.schedule.on(self.date_rules.month_start("SPY"))

# When selection happens, fetch daily bar data from selected assets
self.universe_settings.resolution = Resolution.DAILY

# Add universe selection function
self.add_universe( lambda fundamental: [ ... ] )
```

When a selection should result in no universe changes, you can return the special flag `Universe.UNCHANGED` to return the previously selected universe. This action prevents asset churn in your portfolio and speeds up backtesting.

ETF Constituents Asset Selection

ETFs are baskets of securities seeking to outperform the markets. Many ETFs passively follow popular indexes like the S&P500, providing a cost-efficient way to invest in index constituents with a cost per share 1/100th of the index cost. Indices and overlayed ETFs are also a good list of constituents for stock selection as they follow a well-defined formula for the addition and removal of assets from the universe, which has not changed in many years.

In QuantConnect, you can select an ETF as a data source to power universe selection, closely tracking the index or underlying active strategy. From this universe, you can use the filtering function to request fundamental data of each asset to further narrow your range of instruments.

```
# Add and filter ETF constituents
self.add_universe(self.universe.etf("QQQ", universe_filter_func=self.filter))

# Narrow universe with other data
def filter(self, constituents):
    return [ c.symbol for c in constituents
             if self.fundamentals(c.symbol).valuation_ratios.pe_ratio > 10 ]
```

Dollar-Volume Asset Selection

The simplest and fastest method for universe selection is a price and dollar-volume (price times shares traded) filter. This dataset contains the daily price values and volume traded. Using price information, you can construct any technical indicator required and filter and rank by the most liquid assets available.

QuantConnect has short-cut methods to add this automatically with one line of code. The following automatically selects five assets with the highest dollar-volume from the previous trading date:

```
self.add_universe(self.universe.dollar_volume.top(5))
```

To build more advanced universe types on top of universe data, we recommend using a class to store data and initialize per security properties that keeps your algorithm organized. Using the SymbolData class pattern avoids the common mistake of creating parallel dictionaries to store asset information. As a universe presents its data for selection, the SymbolData constructor can build complex states and track if the asset should be included in the universe.

The following example is pseudocode to share the concept, but the possibilities of each asset's selection and modeling are limitless:

```
class SymbolData:
    # Construct initial state. Use algorithm to setup state
    def __init__(self, algorithm, symbol, period):
        self._symbol = symbol
        self._sma = SimpleMovingAverage(period)
    # Update internal state with required data, train models
    def update(self, time, price, volume):
        self._sma.update(time, volume)
```

Universe Settings

Universes have many configurable settings ([qnt.co/book-universe-selection-settings](https://www.quantconnect.com/docs/v2/quantconnect-api/universe-selection-settings)) to set their desired behavior and control the properties of the assets they add to your strategy. These settings need to be configured during the algorithm initialization. Notable properties that we use in the subsequent chapters are as follows:

Scheduling—Defining a cadence to trigger the universe selection. This improves algorithm performance and reduces asset churn. By default, universes are selected daily.

```
self.universe_settings.schedule.on(date_rule)
```

Extended Market Hours—By default, data feeds contain normal market hours only. By setting this to true, data from pre- and post-market hours will also be passed to your algorithm.

```
self.universe_settings.extended_market_hours = True
```

Data Resolution—Defines the resolution of the asset data added from universe selection.

```
self.universe_settings.resolution = Resolution.HOUR
```

Data Normalization Mode—Defines the data adjustment technique to use for asset data added from selection. By default, full split and dividend data normalization are applied.

```
self.universe_settings.data_normalization_mode = DataNormalizationMode.RAW
```

Indicators and Other Data Transformations

Quant practitioners have created thousands of ways to transform raw market data into more digestible forms for analysis. As these transformations have become more and more sophisticated, their connection with the underlying data has become tenuous. The vast majority of these transformations/filters/indicators are lagging signals—providing a summary of the training averages or volatilities.

There is nothing fundamentally wrong with indicators when viewed as data transformations for the sake of analysis. However, technical analysis tends to cross over into voodoo when *causality* is assigned to technical signals. Technical indicators are useful data transformations but do not *cause* the market to react or move in any way. When applying indicators to your strategy, try and keep an impartial, scientific mindset.

QuantConnect implements hundreds of technical indicators ([qnt.co/book-supported-indicators](https://www.quantconnect.com/docs/v2/faq-fundamentals/technical-indicators)), which are helpers covering most technical and basic financial mathematics analysis. Most technical indicators are just thinly disguised variants of mathematical or financial operations: averages and distributions.

Automatic Indicators

In QuantConnect, there are hundreds of helper functions to create indicators that are immediately ready to use ([qnt.com/book-automatic-indicators](https://www.quantconnect.com/docs/v2/faq-fundamentals/automatic-indicators)) and registered for future data updates automatically. They are represented by the indicator acronym in the API:

```
# Create warmed-up indicator that automatically updates  
auto_rsi = self.rsi("SPY", 10, MovingAverageType.SIMPLE)
```

Manual Indicators

Manual indicators are simple, standalone classes that implement the API behavior. Creating a manual indicator requires manually populating it with required data or registering the object for a data feed. Manually con-

structuring indicators (qnt.co/book-manual-indicators) is harder than automatic ones, but it is helpful to give practitioners total control over the indicators' input data.

```
# Create an indicator that needs manual update
manual_rsi = RelativeStrengthIndex(10, MovingAverageType.SIMPLE)

# Create 3-tradebar consolidator to control data feed for RSI.
consolidator = TradeBarConsolidator(3)

# Register consolidator for data and pipe those 3TB bars to indicator
self.register_indicator(symbol, manual_rsi, consolidator)
```

Indicator Warm Up

QuantConnect has built all its indicators around streaming data feeds to match live trading identically. *Warm up* is the process of streaming historical data into the indicator to prepare it for trading. This can be done automatically by QuantConnect by setting the following flag to true:

```
self.settings.automatic_indicator_warmup = True
```

Storing Objects

You can use Python's duck-typing magic to conveniently store indicator objects on the security object. The following example shows how to create an automatically updated EMA object on Apple's security object. This new property can be accessed anywhere in the algorithm.

```
aapl = self.add_equity("AAPL", Resolution.DAILY) # Create apple security
aapl.ema = self.ema("AAPL", 200) # Assign indicator to duck ema variable
```

Indicator Events

Once an indicator is updated, it fires an event you can use to refresh your algorithm's follow-on calculations. This is useful to control the sequence of events your strategy generates. The following example uses the indicator-updated event to plot the indicator on a chart:

```
# After you create the indicator, attach an event handler
auto_rsi.updated += self.indicator_updated

def update_event_handler(self, indicator, indicator_data_point):
    if indicator.is_ready:
        self.plot("Indicator", "Value", indicator_data_point.value)
```

Sourcing Ideas

The scope of trading ideas a quant can explore is virtually limitless. The markets represent the sum of billions of individual contributing factors in a giant, infinitely complex puzzle! There are tens of thousands of assets, each with their own backgrounds and nuance.

There are two primary camps of idea generation: data-driven investing and hypothesis-driven investing. Both have large established funds following their general principles. We'll explore these two concepts next, as well as online references for how to generate ideas.

Hypothesis-driven Testing

Hypothesis-driven testing focuses on creating a test that you can accept or reject. We recommend that an algorithm hypothesis follow the pattern of cause and effect. Your aim should be to express your strategy in the following sentence:

A change in {cause} leads to an {effect}.

To seek inspiration, consider causes from your experience, intuition, or the media. Generally, causes of financial market movements fall into the following categories:

- Human psychology—mass hysteria or panic, typically the “why” behind momentum (“slow information dissemination” theory).
- World events—core global (political, technological) or local (corporate governance, product releases) events that change the core value of an asset.

- Market or company activity

Consider the examples in [Figure 2.4](#).

FIGURE 2.4

Cause-effect examples for trading strategy ideas.

Cause	leads to	Effect
Share class stocks are the same company, so any price divergence is irrational...	→	A perfect pairs trade. Since they are the same company, the price will revert.
New stock addition to the S&P500 Index causes fund managers to buy up stock...	→	An increase in the price of the new asset in the universe from buying pressure.
Increase in sunshine hours increases the production of oranges...	→	An increase in the supply of oranges and decreased price for orange juice futures.
Allegations of fraud by the CEO causes investor faith in the stock to fall...	→	A collapse of stock prices for the company as people panic.
FDA approval of a new drug for a pharmaceutical company...	→	A jump in stock prices for the company as the market opportunity expands.
News of a data breach potentially exposing the company to millions in liability...	→	A fall in price as investors weigh potential impact.

Ideally, you should form a hypothesis at the start of your research and spend the remaining time exploring how to test your theory. If you find yourself deviating from your core theory or introducing code that isn't based on that hypothesis, you should stop and go back to thesis development as it risks becoming overfit.

Alvarez et al. (2014) illustrated the danger of creating your hypothesis based on test results. In their research, they examined the earnings yield factor in the technology sector over time. During 1998–1999, before the tech bubble burst, the factor was unprofitable. If you had seen the results and then decided to bet against the factor from 2000 to 2002, you would have lost a lot of money because the factor performed extremely well during that time.

There are millions of potential strategies to explore, limited only by your imagination and understanding of the markets. Once you have chosen a strategy to test, we recommend exploring it for a fixed, time-boxed period to reduce the chances of overfitting.

Data Driven Investing

Data driven investing searches for statistical anomalies and attempts to invest in them without an attempt to explain why they may be occurring.

Following this thesis, you believe explaining why an idea happens is largely a waste of time and likely incorrect. Humans are reasoning, pattern-matching machines and can assign reasons on backward-looking events for almost any data.

One of the strongest reasons to use hypothesis-driven investing is that you have a clear signal to stop trading a strategy when the cause of the hypothesis has ended. With data driven investing, funds can use statistically driven methods to remove alphas from the portfolio when they stop performing.

Quantpedia

As a source for inspiration, you can review multiple online sources for ideas. Quantpedia ([qnt.co/book-quantpedia](https://www.quantpedia.com/)) comprehensively compiles and reviews academic research on trading strategies. Its primary goal is

to bridge the gap between academic finance research and practical investment management. They analyze thousands of academic papers to extract actionable trading strategies, covering many asset classes, including equities, commodities, and fixed income.

The platform provides detailed descriptions, performance statistics, and implementation guidelines for each strategy, enabling investors to understand and potentially apply them in real-world scenarios. Strategies are categorized based on factors, such as momentum, value, volatility, and more, allowing users to explore different investment approaches systematically. You can find it at www.quantpedia.com.

QuantConnect Research and Strategy Explorer

The *QuantConnect Research* articles have roughly 100 paper publications with short write-ups on the how the paper was implemented on QuantConnect. These serve as an excellent source of ideas to apply to your own strategies. These can be found at qnt.co/book-research.

The *QuantConnect Explorer* hosts 50 live running strategies written by the core QuantConnect team, running in a public live-trading environment. Each strategy is built to be robust for in- and out-of-sample live trading—fully handling corporate actions and restoring algorithm state in case of restarts. This collection can be found at qnt.co/book-explore.